

Mini Mercado

Documentação Técnica do Código

Análise e Desenvolvimento de Sistemas — 1.º Semestre

1. Estrutura Geral do Projeto

O projeto é dividido em 7 classes Java, cada uma com uma responsabilidade bem definida. Essa separação segue o princípio de coesão: cada arquivo cuida de um único assunto, tornando o código mais fácil de ler, manter e depurar.

Arquivo	Responsabilidade
MiniMercado.java	Ponto de entrada do programa. Contém o main() e o menu principal.
Produto.java	Classe modelo que representa um produto com seus atributos.
Cliente.java	Classe modelo que representa um cliente com seus atributos.
Arquivo.java	Classe utilitária responsável por toda leitura e escrita nos .csv.
MenuProduto.java	Implementa o CRUD completo de produtos (cadastrar, listar, buscar, alterar, remover).
MenuCliente.java	Implementa o CRUD completo de clientes (cadastrar, listar, buscar, alterar, remover).
MenuCompra.java	Gerencia a realização de compras e o controle de estoque.

Os dados persistidos ficam em dois arquivos na pasta dados/:

- produtos.csv — armazena todos os produtos cadastrados
- clientes.csv — armazena todos os clientes cadastrados

2. MiniMercado.java — Ponto de Entrada

Essa é a classe que o Java procura ao executar o programa, pois contém o método `main()`. Ela tem duas responsabilidades: inicializar o sistema carregando os dados salvos anteriormente, e exibir o menu principal em loop até o usuário optar por sair.

Carregamento dos dados

As duas primeiras chamadas dentro do `main()` são:

Inicialização

```
MenuProduto.carregar();
MenuCliente.carregar();
```

Esses métodos leem os arquivos `.csv` e preenchem os `ArrayLists` internos de cada classe antes de qualquer interação com o usuário. Se os arquivos ainda não existirem (primeira execução), os `ArrayLists` ficam vazios e o programa funciona normalmente.

Loop principal com do-while

O menu principal usa uma estrutura `do-while`, garantindo que o menu seja exibido pelo menos uma vez antes de verificar a condição de saída. O loop só termina quando `opcao == 0`.

Estrutura do loop principal

```
int opcao;
do {
    // exibe menu...
    opcao = lerInteiro(sc);
    switch (opcao) {
        case 1 -> MenuProduto.exibir(sc);
        case 2 -> MenuCliente.exibir(sc);
        case 3 -> MenuCompra.realizar(sc);
        case 4 -> MenuCompra.controleEstoque(sc);
        case 0 -> System.out.println("Encerrando...");
        default -> System.out.println("[AVISO] Opção inválida!");
    }
} while (opcao != 0);
```

O `switch` usa a sintaxe de seta (`->`), disponível a partir do Java 14, que dispensa o `break` em cada `case` e torna o código mais limpo.

Método `lerInteiro()`

Esse método auxiliar aparece em todas as classes que precisam ler um número inteiro do teclado. Ele fica em loop infinito (`while true`) e só retorna quando o `Integer.parseInt()` conseguir converter a entrada sem lançar `NumberFormatException`. Se o usuário digitar letras ou deixar em branco, o programa exibe uma mensagem de erro e pede novamente, sem quebrar.

lerInteiro — tratamento de entrada inválida

```
private static int lerInteiro(Scanner sc) {
    while (true) {
        try {
            return Integer.parseInt(sc.nextLine().trim());
        } catch (NumberFormatException e) {
            System.out.print("[ERRO] Digite um número válido: ");
        }
    }
}
```

3. Produto.java — Classe Modelo

A classe Produto representa um único produto do mercado com seus quatro atributos: código (int), nome (String), preço (double) e estoque (int). Ela não contém lógica de menu nem de arquivo — apenas define a estrutura do dado e como ele é convertido para/de texto.

Atributos privados e encapsulamento

Todos os atributos são declarados como `private`, o que impede acesso direto de fora da classe. Para ler ou modificar cada atributo, é necessário passar pelos métodos getters e setters.

Atributos encapsulados

```
private int    codigo;
private String nome;
private double preco;
private int    estoque;
```

Note que não existe `setCodigo()` — o código é definido uma única vez no construtor e nunca mais alterado. Isso evita que o código de um produto seja modificado acidentalmente após o cadastro.

Construtor

O construtor recebe os quatro valores e os atribui aos campos com `this.`, diferenciando os parâmetros do construtor dos atributos da classe que têm o mesmo nome.

Construtor de Produto

```
public Produto(int codigo, String nome, double preco, int estoque) {
    this.codigo = codigo;
    this.nome   = nome;
    this.preco  = preco;
    this.estoque = estoque;
}
```

toCSV() — serialização para arquivo

O método `toCSV()` converte o objeto inteiro em uma String formatada com ponto e vírgula como separador, pronta para ser gravada como uma linha no arquivo `.csv`:

toCSV()

```
public String toCSV() {  
    return codigo + ";" + nome + ";" + preco + ";" + estoque;  
}
```

Exemplo de saída: 1;Coca-Cola 2L;8.99;10

fromCSV() — desserialização do arquivo

O método `fromCSV()` faz o caminho inverso: recebe uma linha do arquivo, divide pelo separador com `split(";")`, converte cada parte para o tipo correto e devolve um objeto `Produto` pronto. É declarado como `static` porque não precisa de uma instância para funcionar — ele cria a instância.

fromCSV()

```
public static Produto fromCSV(String linha) {  
    String[] partes = linha.split(";");  
    int    codigo   = Integer.parseInt(partes[0].trim());  
    String nome     = partes[1].trim();  
    double preco    = Double.parseDouble(partes[2].trim());  
    int    estoque  = Integer.parseInt(partes[3].trim());  
    return new Produto(codigo, nome, preco, estoque);  
}
```

toString() — exibição formatada

O método `toString()` usa `String.format` com `%-Nd` para alinhar o conteúdo em colunas de largura fixa. O sinal de menos (-) indica alinhamento à esquerda, e N é a largura reservada. Isso cria a aparência de tabela no terminal sem usar nenhuma biblioteca externa.

toString()

```
public String toString() {  
    return String.format("| %-5d | %-25s | R$ %-8.2f | %-8d |",  
        codigo, nome, preco, estoque);  
}
```

4. Cliente.java — Classe Modelo

A classe `Cliente` segue exatamente a mesma estrutura de `Produto.java`, com quatro atributos: `cpf (String)`, `nome (String)`, `telefone (String)` e `email (String)`. Como o CPF é o identificador único do cliente, não há `setCpf()` — ele também é imutável após o cadastro.

Os métodos `toCSV()` e `fromCSV()` funcionam da mesma forma: `toCSV()` gera a linha separada por ponto e vírgula, e `fromCSV()` reconstrói o objeto a partir dessa linha. O `toString()` formata a exibição com colunas alinhadas para a listagem no terminal.

5. Arquivo.java — Persistência de Dados

Essa classe centraliza toda a leitura e escrita em disco. Todos os seus métodos são estáticos, o que significa que podem ser chamados diretamente sem criar um objeto Arquivo. Ela possui quatro métodos: `carregarProdutos`, `salvarProdutos`, `carregarClientes` e `salvarClientes`.

Leitura com `BufferedReader`

O carregamento usa `BufferedReader` com `FileReader` para ler o arquivo linha por linha de forma eficiente. A primeira linha é sempre o cabeçalho (como `codigo;nome;preco;estoque`) e é ignorada com uma flag booleana. Cada linha seguinte é passada para o `fromCSV()` da classe correspondente, que devolve um objeto pronto para ser inserido no `ArrayList`.

`carregarProdutos()`

```
try (BufferedReader br = new BufferedReader(new FileReader(arquivo))) {
    String linha;
    boolean primeiraLinha = true;
    while ((linha = br.readLine()) != null) {
        if (primeiraLinha) { primeiraLinha = false; continue; }
        if (!linha.isBlank()) {
            lista.add(Produto.fromCSV(linha));
        }
    }
}
```

O verificador `linha.isBlank()` garante que linhas vazias no final do arquivo não causem erros de parsing.

Escrita com `PrintWriter`

O salvamento usa `PrintWriter` com `FileWriter` passando `false` como segundo argumento. Esse `false` indica que o arquivo deve ser sobrescrito do zero a cada salvamento, não que o conteúdo novo seja acrescentado ao final. O cabeçalho é escrito primeiro, e depois um loop percorre o `ArrayList` chamando `toCSV()` em cada objeto.

`salvarProdutos()`

```
try (PrintWriter pw = new PrintWriter(new FileWriter(caminho, false))) {
    pw.println("codigo;nome;preco;estoque");
    for (Produto p : lista) {
        pw.println(p.toCSV());
    }
}
```

Try-with-resources

Ambos os métodos usam a sintaxe `try (recurso = ...) {}`. Esse padrão, chamado try-with-resources, garante que o arquivo seja fechado automaticamente ao fim do bloco, mesmo que ocorra uma exceção. Sem isso, seria necessário fechar manualmente com `br.close()` em um bloco `finally`, o que é mais verboso e sujeito a erros.

Verificação de existência do arquivo

Antes de tentar abrir o arquivo para leitura, o código verifica se ele existe com `arquivo.exists()`. Se não existir (primeira execução do sistema), o método simplesmente retorna um `ArrayList` vazio. Isso evita uma `FileNotFoundException` e deixa o programa iniciar normalmente sem nenhum dado pré-existente.

6. MenuProduto.java — CRUD de Produtos

Essa classe gerencia todas as operações sobre produtos. O `ArrayList` de produtos é declarado como atributo estático da própria classe, o que significa que ele existe uma única vez na memória durante toda a execução do programa — qualquer método que o modifique está modificando a mesma lista.

Atributos estáticos da classe

```
private static final String ARQUIVO = "../dados/produtos.csv";
private static ArrayList<Produto> produtos = new ArrayList<>();
```

Menu com do-while e switch

O método `exibir()` usa um `do-while` para manter o submenu ativo até o usuário digitar 0. O `switch` encaminha para o método correto conforme a opção escolhida. A lógica é a mesma do menu principal em `MiniMercado.java`.

Cadastrar — geração automática de código

O código do produto é gerado pelo método `gerarProximoCodigo()`, que percorre o `ArrayList` em busca do maior código existente e retorna esse valor mais 1. Dessa forma o usuário nunca precisa informar o código manualmente, e não há risco de duplicidade.

gerarProximoCodigo()

```
private static int gerarProximoCodigo() {
    int max = 0;
    for (Produto p : produtos) {
        if (p.getCodigo() > max) max = p.getCodigo();
    }
    return max + 1;
}
```

As entradas de preço e estoque são lidas como `String` e convertidas manualmente, o que permite capturar erros de formato com `try-catch` antes de atribuir o valor. O preço aceita tanto ponto quanto vírgula como separador decimal, por conta do `replace(",", ".")`.

Buscar — por código ou nome parcial

A busca percorre o `ArrayList` inteiro e exibe qualquer produto cujo código coincida exatamente com o termo digitado, ou cujo nome contenha o termo como substring (busca parcial case-insensitive). Isso significa que digitar "cola" retornará "Coca-Cola", "Cola Zero" etc.

Lógica de busca

```
if (String.valueOf(p.getCodigo()).equals(termo) ||
    p.getNome().toLowerCase().contains(termo)) {
    System.out.println(p);
}
```

Alterar — campos opcionais

A alteração de produto permite que o usuário pressione ENTER sem digitar nada para manter o valor atual de cada campo. Isso é feito verificando se a String lida está vazia antes de chamar o setter. Assim o usuário não precisa redigitar nome e preço só para alterar o estoque, por exemplo.

Alterar campo opcionalmente

```
System.out.print("Novo nome (ENTER para manter): ");
String nome = sc.nextLine().trim();
if (!nome.isEmpty()) encontrado.setNome(nome);
```

Remover — confirmação antes de excluir

Antes de remover um produto do ArrayList, o sistema exibe o nome do produto e pede confirmação S/N. Apenas se a resposta for exatamente "S" (após toUpperCase()) a remoção é executada e o arquivo é salvo. Isso evita exclusões acidentais por um dígito errado.

encontrarPorCodigo() — método público

Esse método percorre o ArrayList e retorna o objeto Produto cujo código corresponde ao parâmetro, ou null se não encontrar. É declarado como public porque é chamado também por MenuCompra ao montar e confirmar uma compra.

7. MenuCliente.java — CRUD de Clientes

A estrutura de MenuCliente.java é análoga à de MenuProduto.java, com um ArrayList estático de clientes, os mesmos padrões de menu, busca e alteração opcional de campos. As diferenças estão nas validações específicas de cliente.

Validação e limpeza de CPF

Ao cadastrar ou buscar por CPF, o código remove todos os caracteres que não sejam dígitos com a expressão regular `[^0-9]`. Isso permite que o usuário digite o CPF em qualquer formato (com pontos e traço, ou só números) sem que o sistema rejeite a entrada.

Limpeza do CPF

```
String cpf = sc.nextLine().trim().replaceAll("[^0-9]", "");
if (cpf.length() != 11) {
    System.out.println("[ERRO] CPF inválido. Informe 11 dígitos numéricos.");
    return;
}
```

Unicidade do CPF

Antes de inserir um novo cliente, o método `encontrarPorCpf()` é chamado para verificar se o CPF já está cadastrado. Se retornar um objeto diferente de null, o cadastro é recusado com mensagem de erro, garantindo que não existam dois clientes com o mesmo CPF.

`encontrarPorCpf()` — método público

Esse método é público porque é chamado por `MenuCompra` durante o processo de compra para identificar o cliente pelo CPF digitado. Ele também normaliza o CPF recebido com `replaceAll` antes de comparar, para consistência.

8. MenuCompra.java — Compras e Estoque

Essa é a classe mais complexa do sistema. O método `realizar()` implementa o fluxo completo de uma compra: identificação do cliente, montagem do carrinho, validação de estoque, exibição do cupom e débito do estoque.

Carrinho com vetores paralelos

O carrinho de compras é implementado com quatro vetores que trabalham em sincronia pelo índice. Cada posição `i` representa um item diferente na compra:

Vetores paralelos do carrinho

```
int[]    codigosCarrinho    = new int[100];
int[]    quantCarrinho     = new int[100];
double[] subtotaisCarrinho  = new double[100];
String[] nomesCarrinho     = new String[100];
int      totalItens        = 0;
```

A variável `totalItens` controla quantas posições estão efetivamente ocupadas. Ao adicionar um produto novo, ele é inserido na posição `totalItens` e o contador é incrementado. Ao exibir o cupom, o loop vai de 0 até `totalItens - 1`.

Validação de estoque com reserva

A validação de estoque considera não apenas o estoque atual no `ArrayList`, mas também o que já foi reservado no carrinho durante a mesma compra. Isso impede que o usuário adicione o mesmo produto em duas interações separadas e ultrapasse o estoque disponível.

Cálculo do estoque real disponível

```
int jaNoCarrinho = 0;
for (int i = 0; i < totalItens; i++) {
    if (codigosCarrinho[i] == codigo) {
        jaNoCarrinho = quantCarrinho[i];
        break;
    }
}
int disponivelReal = produto.getEstoque() - jaNoCarrinho;
```



```

if (quantidade > disponivelReal) {
    System.out.println("[ERRO] Estoque insuficiente! Disponível: " +
        disponivelReal);
    continue;
}

```

Produto repetido no carrinho

Se o usuário adicionar o mesmo produto duas vezes, o sistema detecta isso percorrendo os vetores em busca do mesmo código. Se encontrar, soma a quantidade e recalcula o subtotal naquela mesma posição, em vez de criar uma entrada duplicada. O booleano `jaAdicionado` controla esse fluxo.

Detecção de produto duplicado no carrinho

```

boolean jaAdicionado = false;
for (int i = 0; i < totalItens; i++) {
    if (codigosCarrinho[i] == codigo) {
        quantCarrinho[i] += quantidade;
        subtotaisCarrinho[i] = quantCarrinho[i] * produto.getPreco();
        jaAdicionado = true;
        break;
    }
}
if (!jaAdicionado) {
    codigosCarrinho[totalItens] = codigo;
    nomesCarrinho[totalItens] = produto.getNome();
    quantCarrinho[totalItens] = quantidade;
    subtotaisCarrinho[totalItens] = quantidade * produto.getPreco();
    totalItens++;
}

```

Cupom fiscal

O cupom usa `System.out.printf` com formatação `%-20s %5d R$%7.2f` para alinhar as colunas. O `%7.2f` reserva 7 caracteres para o número com 2 casas decimais, garantindo que os valores monetários fiquem alinhados verticalmente.

Débito no estoque após confirmação

Somente após o usuário confirmar a compra com "S" o estoque é debitado. O loop percorre os vetores do carrinho, localiza cada Produto pelo código e chama `setEstoque()` com o novo valor. Por fim, `MenuProduto.salvar()` é chamado para persistir as alterações no arquivo CSV.

Débito no estoque

```

for (int i = 0; i < totalItens; i++) {
    Produto p = MenuProduto.encontrarPorCodigo(codigosCarrinho[i]);
    if (p != null) {
        p.setEstoque(p.getEstoque() - quantCarrinho[i]);
    }
}
MenuProduto.salvar();

```

Controle de Estoque

O submenu de controle de estoque oferece três operações adicionais. A listagem de produtos com estoque baixo percorre o ArrayList e filtra apenas os que possuem estoque menor que 5, exibindo um alerta. A reposição localiza o produto pelo código e usa setEstoque() para somar a quantidade informada ao valor atual, salvando em seguida.

9. Conceitos da Disciplina Aplicados

Conceito	Onde e como foi aplicado
Variáveis e tipos	int, double, String, boolean declarados e usados em todos os arquivos
Condicionais (if/switch)	Validações de entrada nos CRUDs; switch nos menus principal e submenus
do-while	Mantém os menus ativos até o usuário digitar 0
while	Loop do carrinho de compras; loop do lerInteiro()
for / for-each	Percorre ArrayLists nas listagens, buscas e no débito de estoque
Vetores	Quatro vetores paralelos representam o carrinho em MenuCompra
Modularização	Cada operação é um método privado; métodos utilitários reutilizados
Classes e objetos	Produto e Cliente instanciados com new; atributos, construtor, getters, setters
static	Atributos e métodos estáticos nas classes de menu; fromCSV() em modelos
Persistência CSV	BufferedReader para leitura, PrintWriter para escrita em Arquivo.java
try-catch	Captura NumberFormatException nas entradas; IOException nas operações de arquivo
try-with-resources	Fecha BufferedReader e PrintWriter automaticamente em Arquivo.java